

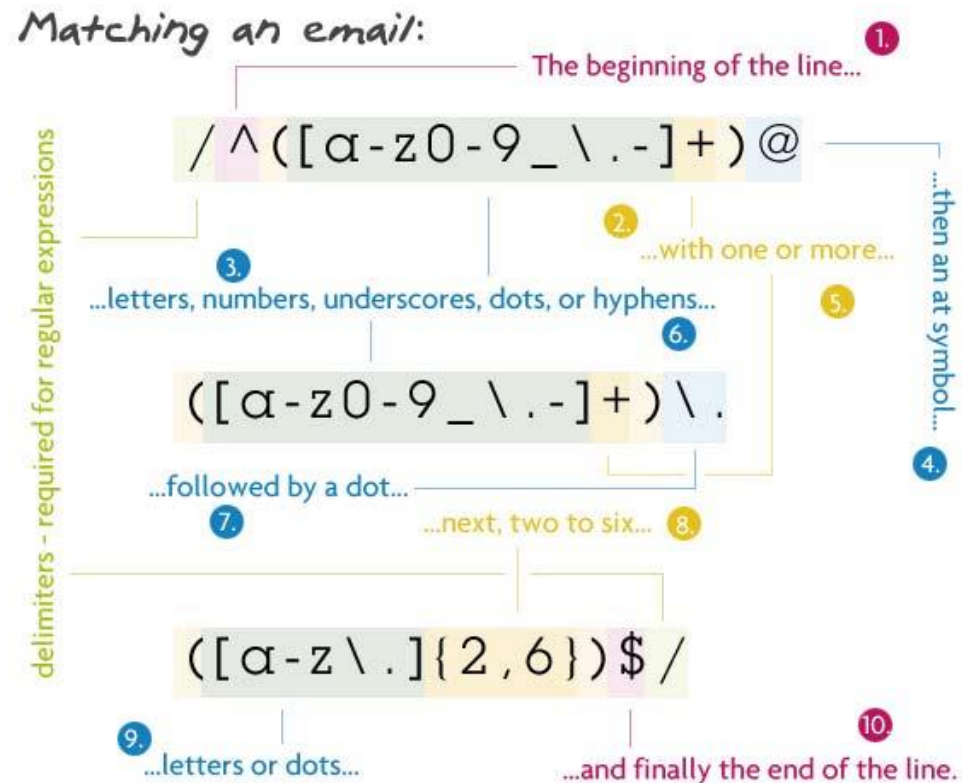


API para datas



Regular Expressions

Expressões regulares permitem-nos processar texto, identificando padrões sofisticados, baseando-se em Álgebra. Encontra-se de forma comum em editores de texto (interpretadores de linguagens), mas também em processamento de passwords (regras de letras maiúsculas, símbolos, números...). Dependendo do que precisamos, pode tornar-se uma operação bastante complexa rapidamente. O Java fornece-nos estas funcionalidades através do package `java.util.regex`.





DateFormat

O package `java.text` permite-nos formatar, tratar, pesquisar e manipular texto. A partir do JDK 9, este package está integrado no módulo `java.base`. Uma das classes que aqui encontramos é a `DateFormat`. Trata-se de uma classe abstrata que permite formatar e tratar datas e horas. `getInstance()` retorna uma instância de `DateFormat` que pode formatar informação de datas e podemos utilizar da seguinte forma.

```
static final DateFormat getInstance( )  
static final DateFormat getInstance(int style)  
static final DateFormat getInstance(int style, Locale locale)
```

Style é um dos seguintes valores: `DEFAULT`, `SHORT`, `MEDIUM`, `LONG` ou `FULL`. Estas variáveis são ints definidos por `DateFormat`. O output vai diferir de acordo com estes parâmetros, uma vez que dizem respeito à forma de apresentação dos mesmos. Locale diz respeito às regras de localização.



DateFormat

Um dos métodos mais utilizados desta classe é o format():

```
// Demonstrate date formats.
import java.text.*;
import java.util.*;

public class DateFormatDemo {
    public static void main(String args[]) {
        Date date = new Date();
        DateFormat df;

        df = DateFormat.getDateInstance(DateFormat.SHORT, Locale.JAPAN);
        System.out.println("Japan: " + df.format(date));

        df = DateFormat.getDateInstance(DateFormat.MEDIUM, Locale.KOREA);
        System.out.println("Korea: " + df.format(date));

        df = DateFormat.getDateInstance(DateFormat.LONG, Locale.UK);
        System.out.println("United Kingdom: " + df.format(date));

        df = DateFormat.getDateInstance(DateFormat.FULL, Locale.US);
        System.out.println("United States: " + df.format(date));
    }
}
```

Japan: 2018/06/20
Korea: 2018. 6. 20.
United Kingdom: 20 June 2018
United States: Wednesday, June 20, 2018



DateFormat

À semelhança da data, também podemos obter uma instância de DateFormat para representar as horas. O método getInstance usa-se das seguintes formas:

```
static final DateFormat getInstance( )  
static final DateFormat getInstance(int style)  
static final DateFormat getInstance(int style, Locale locale)
```

Style tem as mesmas opções que para a data: DEFAULT, SHORT, MEDIUM, LONG e FULL.



DateFormat

```
// Demonstrate time formats.
import java.text.*;
import java.util.*;
public class TimeFormatDemo {
    public static void main(String args[]) {
        Date date = new Date();
        DateFormat df;

        df = DateFormat.getInstance(DateFormat.SHORT, Locale.JAPAN);
        System.out.println("Japan: " + df.format(date));

        df = DateFormat.getInstance(DateFormat.LONG, Locale.UK);
        System.out.println("United Kingdom: " + df.format(date));

        df = DateFormat.getInstance(DateFormat.FULL, Locale.CANADA);
        System.out.println("Canada: " + df.format(date));
    }
}
```

Japan: 13:03
United Kingdom: 13:03:31 GMT-06:00
Canada: 1:03:31 PM Central Daylight Time



SimpleDateFormat

Por vezes podemos precisar de utilizar um formato personalizado, não suportado pela DateFormat. Para isso, o JDK tem a SimpleDateFormat, subclasse de DateFormat, que nos permite definir os nossos próprios padrões para mostrar data e hora. Um dos seus construtores é:

`SimpleDateFormat(String formatString)`

`formatString` define a formatação que queremos, um pouco à semelhança daquilo que são as Regular Expressions (Regex). Um exemplo de utilização:

```
SimpleDateFormat sdf = SimpleDateFormat("dd MMM yyyy hh:mm:ss  
zzz");
```



SimpleDateFormat

Symbol	Description
a	AM or PM
d	Day of month (1–31)
h	Hour in AM/PM (1–12)
k	Hour in day (1–24)
m	Minute in hour (0–59)
s	Second in minute (0–59)
u	Day of week, with Monday being 1
w	Week of year (1–52)
y	Year
z	Time zone
D	Day of year (1–366)
E	Day of week (for example, Thursday)
F	Day of week in month
G	Era (for example, AD or BC)
H	Hour in day (0–23)
K	Hour in AM/PM (0–11)
L	Month
M	Month
S	Millisecond in second
W	Week of month (1–5)
X	Time zone in ISO 8601 format
Y	Week year
Z	Time zone in RFC 822 format



SimpleDateFormat

O número de vezes que cada caractere está repetido na String determina como os dados são apresentados. Se este número for inferior a 4, então o texto será abreviado. 'zzzz' pode mostrar 'Pacific Daylight Time' enquanto 'zzz' mostra PDT.

Para números, o mesmo também acontece mas com algumas diferenças. hh:mm:ss pode representar 01:51:15, enquanto h:m:s mostra 1:51:15. M ou MM demonstrará o mês com 1 ou 2 dígitos (obrigatoriamente 2 para os últimos 3 meses do ano), MMM representará o mês em texto, apenas com as primeiras 3 letras enquanto MMMM representará o mês por extenso.



SimpleDateFormat

```
// Demonstrate SimpleDateFormat.
import java.text.*;
import java.util.*;

public class SimpleDateFormatDemo {
    public static void main(String args[]) {
        Date date = new Date();
        SimpleDateFormat sdf;
        sdf = new SimpleDateFormat("hh:mm:ss");
        System.out.println(sdf.format(date));
        sdf = new SimpleDateFormat("dd MMM yyyy hh:mm:ss zzz");
        System.out.println(sdf.format(date));
        sdf = new SimpleDateFormat("E MMM dd yyyy");
        System.out.println(sdf.format(date));
    }
}
```

01:30:51

20 Jun 2018 01:30:51 CDT

Wed Jun 20 2018



Java.time

Este módulo foi integrado no JDK 8, com a intenção de gerir todo o tratamento temporal de uma outra forma, uma vez que há diferentes calendários e formas de contagem alternativas.

No java.time temos várias classes que nos dão acesso fácil à data e hora. `LocalDate`, `LocalTime` e `LocalDateTime` são três exemplos disso. São baseadas em valores e, como o nome sugere, encapsulam a data, a hora e a data e hora. Ao usá-las podemos facilmente obter, formatar e comparar datas e horas entre outras operações.

`LocalDate` encapsula uma data, usando o calendário Gregoriano por defeito como especificado em ISO 8601. `LocalTime` encapsula a hora, respeitando também a mesma regra descrita em ISO 8601. `LocalDateTime` faz o mesmo, em ambos. Estas classes contêm um grande número de métodos para dar acesso aos seus componentes, permitindo compará-los, fazer operações aritméticas, entre outros.

Uma função que nos permite instanciar qualquer uma destas classes (uma vez que não têm construtores públicos) é `now()`. Retorna-nos a data e hora atual do sistema. Cada classe tem a sua versão, mas para efeito de exemplo vamos usar a sua forma mais simples.

```
static LocalDate now()  
static LocalTime now()  
static LocalDateTime now()
```



Java.time

Em cada um dos exemplos do slide anterior, um objeto é retornado, podendo ser imediatamente usado num `println()`, por exemplo. Também podemos aqui controlar a sua formatação.

O seguinte código utiliza `LocalDate` e `LocalTime` para obter a data e hora correntes e depois mostrá-los. Reparem como `now()` é chamado.

```
// A simple example of LocalDate and LocalTime.  
import java.time.*;
```

```
class DateTimeDemo {  
    public static void main(String args[]) {  
  
        LocalDate curDate = LocalDate.now();  
        System.out.println(curDate);  
        LocalTime curTime = LocalTime.now();  
        System.out.println(curTime);  
    }  
}
```

```
2018-06-20  
17:31:15.274937600
```

```
LocalDateTime curDateTime =  
    LocalDateTime.now();  
System.out.println(curDateTim  
e);
```

```
2018-06-20T17:34:18.991974600
```

Tokio.



Java.time

Também podemos converter LocalDateTime para cada uma das outras duas classes, através dos seguintes métodos:

LocalDate toLocalDate()

LocalTime toLocalTime()



Formatar data e hora

Com estas classes também podemos formatar data e hora caso pretendamos que estes sejam mostrados de uma forma diferente, através do comando `format`:

```
String format(DateTimeFormatter fmtr)
```

`fmtr` especifica a instância de `DateTimeFormatter` que disponibiliza o formato. Esta classe encontra-se em `java.time.format`. Para obtermos uma instância vamos usar um dos seus métodos de fábrica.

```
static DateTimeFormatter ofLocalizedDate(FormatStyle fmtDate)
```

```
static DateTimeFormatter ofLocalizedTime(FormatStyle fmtTime)
```

```
static DateTimeFormatter ofLocalizedDateTime(FormatStyle fmtDate,  
                                              FormatStyle fmtTime)
```



Formatar data e hora

Naturalmente, cada um dos métodos anteriores vai devolver uma instância adequada à classe que estamos a operar. `FormatStyle` é um enum em `java.time.format` cujas opções são `FULL`, `LONG`, `MEDIUM` e `SHORT`. Especificam o nível de detalhe mostrado, semelhante a `java.text.DateFormat`.

```
// Demonstrate DateTimeFormatter.
import java.time.*;
import java.time.format.*;

class DateTimeDemo2 {
    public static void main(String args[]) {

        LocalDate curDate = LocalDate.now();
        System.out.println(curDate.format(
            DateTimeFormatter.ofLocalizedDate(FormatStyle.FULL)));

        LocalTime curTime = LocalTime.now();
        System.out.println(curTime.format(
            DateTimeFormatter.ofLocalizedTime(FormatStyle.SHORT)));
    }
}
```

Wednesday, June 20, 2018
2:16 PM



Formatar data e hora

Se quisermos um padrão diferente, podemos obtê-lo também de forma semelhante ao que víamos em `DateFormat`, usando o método:

```
static DateTimeFormatter ofPattern(String fmtPattern)
```

`fmtPattern` é uma string que contém os padrões de data e hora que pretendemos. Retorna um `DateTimeFormatter` que formatará de acordo com o que lhe damos. O locale por defeito é usado.

Em geral, um padrão consiste em especificadores de formato. Uma letra padrão será substituída pelo componente da data ou hora correspondente. No próximo slide temos uma tabela com estas letras.



Formatar data e hora

a	AM/PM indicator
d	Day in month
E	Day in week
h	Hour, 12-hour clock
H	Hour, 24-hour clock
M	Month
m	Minutes
s	Seconds
y	Year

M MM MMM MMMM

4 04 Apr April



Formatar data e hora

```
// Create a custom date and time format.  
import java.time.*;  
import java.time.format.*;  
  
class DateTimeDemo3 {  
    public static void main(String args[]) {  
  
        LocalDateTime curDateTime = LocalDateTime.now();  
        System.out.println(curDateTime.format(  
            DateTimeFormatter.ofPattern("MMMM d', ' yyyy h':'mm a"))));  
    }  
}
```

June 20, 2018 2:22 PM



Formatar data e hora

Cada uma das classes `LocalTime`, `LocalDate` e `LocalDateTime` permitem obter cada um dos seus componentes através de métodos. Por exemplo, podemos usar `getHour()` para obter um `int` com as horas, `getMonth()` para obter um `enum` do tipo `Month`, `getYear()` para obter um `int` com o ano... Através destes métodos também podemos construir o nosso próprio output.



Formatar data e hora

As 3 classes também permitem fazer parse a strings. Para o fazermos, usamos o método `parse()` numa instância destas classes, como demonstrado.

```
// Parse a date and time.
import java.time.*;
import java.time.format.*;

class DateTimeDemo4 {
    public static void main(String args[]) {

        // Obtain a LocalDateTime object by parsing a date and time string.
        LocalDateTime curDateTime =
            LocalDateTime.parse("June 21, 2018 12:01 AM",
                DateTimeFormatter.ofPattern("MMMM d', ' yyyy hh': 'mm a"));

        // Now, display the parsed date and time.
        System.out.println(curDateTime.format(
            DateTimeFormatter.ofPattern("MMMM d', ' yyyy h': 'mm a")));
    }
}
```